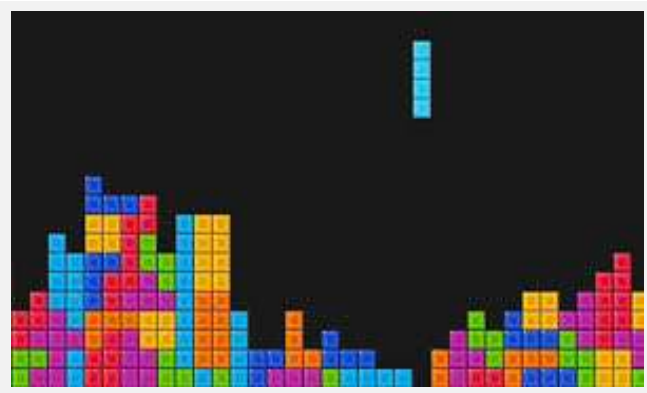

Figure 3.1.5 The board of the tetris game. The squares fill up as blocks of different shapes fall from the top of the grid and are added to the top of the wall. The shapes are generated according to some stochastic process. As a given block falls, the player can move horizontally and rotate the block in all possible ways, subject to the constraints imposed by the sides of the grid and the top of the wall. When a row of full squares is created, this row is removed, the bricks lying above this row move one row downward, and the player scores a point. The player's objective is to maximize the score attained (total number of rows removed) within N steps or up to termination of the game, whichever occurs first.

Example 3.1.3 (Tetris)

Let us consider the game of tetris, which we formulated in Example 1.6.2 as a stochastic shortest path problem with the termination state being the end of the game (see Fig. 3.1.5). The state is the pair of the board position x and the shape of the current falling block y. We viewed as control, the horizontal positioning and rotation applied to the falling block. The optimal cost-to-go function is a vector of huge dimension (there are 2^{200} board positions in a "standard" tetris board of width 10 and height 20). However, it has been successfully approximated in practice by low-dimensional linear architectures.

In particular, the following features have been proposed in the paper by Bertsekas and Ioffe [BeI96]: the heights of the columns, the height differentials of adjacent columns, the wall height (the maximum column height), the number of holes of the board, and the constant 1 (the unit is often included as a feature in cost approximation architectures, as it allows for a constant shift in the approximating function). These features are readily recognized by tetris players as capturing important aspects of the board position.† There

† The use of feature-based approximate DP methods for the game of tetris was first suggested in the paper by Tsitsiklis and Van Roy [TsV96], which introduced just two features (in addition to the constant 1): the wall height and the number of holes of the board. Most studies have used the set of features of [BeI96] described here, but other sets of features have also been used; see [ThS09] and the discussion in [GGS13].

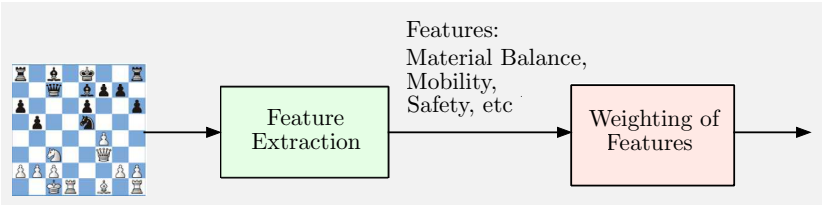

Figure 3.1.6 A feature-based architecture for computer chess.

are a total of 22 features for a "standard" board with 10 columns. Of course the $2^{200} \times 22$ matrix of feature values cannot be stored in a computer, but for any board position, the corresponding row of features can be easily generated, and this is sufficient for implementation of the associated approximate DP algorithms. For recent works involving approximate DP methods and the preceding 22 features, see [Sch13], [GGS13], and [SGG15], which reference several other related papers.

In the works mentioned above the shapes of the falling blocks are stochastically independent. In a more challenging version of the problem, which has not been considered in the literature thus far, successive shapes are correlated. Then the state of the problem would become more complex, since past shapes would be useful in predicting future shapes. As a result, we may need to introduce state estimation and additional features in order to properly deal with the effects of correlations.

Example 3.1.4 (Computer Chess)

Computer chess programs that involve feature-based architectures have existed for many years, and are still used widely (they have been upstaged in the mid-2010s by alternative types of chess programs, which use neural network techniques that will be discussed later). These programs are based on approximate DP for minimax problems, a feature-based parametric architecture, and multistep lookahead.

The fundamental principles on which all computer chess programs (as well as most two-person game programs) are based were laid out by Shannon [Sha50], before Bellman started his work on DP. Shannon proposed multistep lookahead and evaluation of the end positions by means of a "scoring function" (in our terminology this plays the role of a cost function approximation). This function may involve for example the calculation of a numerical value for each of a set of major features of a position that chess players easily recognize (such as material balance, mobility, pawn structure, and other positional factors), together with a method to combine these numerical values into a single score. Shannon then went on to describe various strategies of exhaustive and selective search over a multistep lookahead tree of moves.

We may view the scoring function as a feature-based architecture for evaluating a chess position/state (cf. Fig. 3.1.6). In most computer chess programs, the features are weighted linearly, i.e., the architecture $\tilde{J}(x, r)$ that is used for multistep lookahead is linear [cf. Eq. (3.3)]. In many cases, the weights have been determined manually, by trial and error based on experi-

ence. However, in some programs, the weights have been determined with supervised learning techniques that use examples of grandmaster play, i.e., by adjustment to bring the play of the program as close as possible to the play of chess grandmasters. This is a technique that applies more broadly in artificial intelligence; see Tesauro [Tes89b], [Tes01].

In a recent computer chess breakthrough, the entire idea of extracting features of a position through human expertise was abandoned in favor of feature discovery through self-play and the use of neural networks. The first program of this type to attain supremacy over humans, as well as over the best computer programs that use human expertise-based features, was AlphaZero (Silver et al. [SHS17]). This program, described in Section 1.1, is based on DP principles of approximate policy iteration and multistep lookahead based on Monte Carlo tree search.

Our next example relates to a methodology for feature construction, where the number of features may increase as more data is collected. For a simple example, consider the piecewise constant approximation of Example 3.1.1, where more pieces are progressively added based on new data, possibly using some form of exploration-exploitation tradeoff.

Example 3.1.5 (Feature Extraction from Data)

We have viewed so far feature vectors $\phi(x)$ as functions of x, obtained through some unspecified process that is based on prior knowledge about the cost function being approximated. On the other hand, features may also be extracted from data. For example suppose that with some preliminary calculation using data, we have identified some suitable states $x(\ell)$, $\ell = 1, \ldots, m$, that can serve as "anchors" for the construction of Gaussian basis functions of the form

$$\phi_\ell(x) = e^{-\frac{\|x - x(\ell)\|^2}{2\sigma^2}}, \qquad \ell = 1, \ldots, m, \tag{3.4}$$

where σ is a scalar "variance" parameter, and $\|\cdot\|$ denotes the standard Euclidean norm. This type of function is known as a *radial basis function*. It is concentrated around the state $x(\ell)$, and it is weighed with a scalar weight r_ℓ to form a parametric linear feature-based architecture, which can be trained using additional data. Several other types of data-dependent basis functions, such as support vector machines, are used in machine learning, where they are often referred to as *kernels*.

While it is possible to use a preliminary calculation to obtain the anchors $x(\ell)$ in Eq. (3.4), and then use additional data for training, one may also consider enrichment of the set of basis functions simultaneously with training. In this case the number of the basis functions increases as the training data is collected. A motivation here is that the quality of the approximation may increase with additional basis functions. This idea underlies a field of machine learning, known as *kernel methods* or sometimes *nonparametric methods*.

A further discussion is outside our scope. We refer to the literature; see e.g., books such as Cristianini and Shawe-Taylor [ChS00], [ShC04], Scholkopf and Smola [ScS02], Bishop [Bis06], Kung [Kun14], surveys such as Hofmann,

Scholkopf, and Smola [HSS08], Pillonetto et al. [PDC14], RL-related discussions such as Dietterich and Wang [DiW02], Ormoneit and Sen [OrS02], Engel, Mannor, and Meir [EMM05], Jung and Polani [JuP07], Reisinger, Stone, and Miikkulainen [RSM08], Busoniu et al. [BBD10a], Bethke [Bet10], and recent developments such as Tu et al. [TRV16], Rudi, Carratino, and Rosasco [RCR17], Belkin, Ma, and Mandal [BMM18]. In what follows, for the sake of simplicity, we will focus on parametric architectures with a fixed and given feature vector, since the choice of approximation architecture is somewhat peripheral to our main focus.

The next example considers a feature extraction strategy that is particularly relevant to problems of partial state information.

Example 3.1.6 (Feature Extraction from Sufficient Statistics)

The concept of a sufficient statistic, which originated in inference methodologies, plays an important role in DP. As discussed in Section 1.6, it refers to quantities that summarize all the essential content of the state x_k for optimal control selection at time k.

In particular, consider a partial information context where at time k we have accumulated the *information vector* (also called the *past history*)

$$I_k = (z_0, \ldots, z_k, u_0, \ldots, u_{k-1}),$$

which consists of the past controls $u_0, \ldots, u_{k-1}$ and the state-related measurements $z_0, \ldots, z_k$ obtained at the times $0, \ldots, k$. The control u_k is allowed to depend only on I_k, and the optimal policy is a sequence of the form $\big\{\mu_0^*(I_0), \ldots, \mu_{N-1}^*(I_{N-1})\big\}$. We say that a function $S_k(I_k)$ is a *sufficient statistic at time* k if the control function μ_k^* depends on I_k only through $S_k(I_k)$, i.e., for some function $\hat{\mu}_k$, we have

$$\mu_k^*(I_k) = \hat{\mu}_k\big(S_k(I_k)\big),$$

where μ_k^* is optimal.

There are several examples of sufficient statistics, and they are typically problem-dependent. A trivial possibility is to view I_k itself as a sufficient statistic, and a more sophisticated possibility is to view the *belief state* b_k as a sufficient statistic (this is the conditional probability distribution of x_k given I_k; cf. Section 1.6.6). For a proof that b_k is indeed a sufficient statistic and for a more detailed discussion of other possible sufficient statistics, see [Ber17a], Chapter 4. For a mathematically more advanced discussion, see [BeS78], Chapter 10.

Since a sufficient statistic contains all the relevant information for optimal control purposes, it is natural to introduce features of a given sufficient statistic and to train a corresponding approximation architecture accordingly. Potentially effective features could include some special characteristic of I_k (such as whether some alarm-like "special" event has been observed), or a partial history (such as the last m measurements and controls in I_k, or more sophisticated versions based on the concept of a *finite-state controller* proposed

by White [Whi91], and White and Scherer [WhS94], and further discussed by Hansen [Han98], Kaelbling, Littman, and Cassandra [KLC98], Meuleau et al. [MPK99], Poupart and Boutilier [PoB04], Yu and Bertsekas [YuB08], Saldi, Yuksel, and Linder [SYL17]). In the case where the belief state b_k is used as a sufficient statistic, examples of good features may be a point estimate based on b_k, the variance of this estimate, and other quantities that can be simply extracted from b_k.

The paper by Bhattacharya et al. [BBW20] considers another type of feature vector that is related to the belief state. This is a sufficient statistic, denoted by y_k, which subsumes the belief state b_k, in the sense that b_k can be computed exactly knowing y_k. One possibility is for y_k to be the union of b_k and some identifiable characteristics of the belief state, or some compact representation of the measurement history up to the current time (such as a number of most recent measurements, or the state of a finite-state controller). Even though the information content of y_k is no different than the information content of b_k for the purposes of exact optimization, a sufficient statistic y_k that is specially designed for the problem at hand may lead to improved performance in the presence of cost and policy approximations.

We finally note a related idea, which is to supplement a sufficient statistic with features of other sufficient statistics, and thus obtain an enlarged/richer sufficient statistic. In problem-specific contexts, and in the presence of approximations, this may yield improved results.

Example 3.1.7 (Feature-Based Dimensionality Reduction by Aggregation)

The use of a feature vector $\phi(x)$ to represent the state x in an approximation architecture of the form $\tilde{J}\big(\phi(x), r\big)$ implicitly involves *state aggregation*, i.e., the grouping of states into subsets. We will discuss aggregation in some detail in Section 3.6. Here we will give a summary of a special type of aggregation architecture.

In particular, let us assume that the feature vector can take only a finite number of values, and define for each possible value v, the subset of states S_v whose feature vector is equal to v:

$$S_v = \big\{x \mid \phi(x) = v\big\}.$$

We refer to the sets S_v as the *aggregate states* induced by the feature vector. These sets form a partition of the state space. An approximate cost-to-go function of the form $\tilde{J}\big(\phi(x), r\big)$ is piecewise constant with respect to this partition; that is, it assigns the same cost-to-go value $\tilde{J}(v, r)$ to all states in the set S_v.

An often useful approach to deal with problem complexity in DP is to introduce an "aggregate" DP problem, whose states are some suitably defined feature vectors $\phi(x)$ of the original problem. The precise form of the aggregate problem may depend on intuition and/or heuristic reasoning, based on our understanding of the original problem. Suppose now that the aggregate problem is simple enough to be solved exactly by DP, and let $\hat{J}(v)$

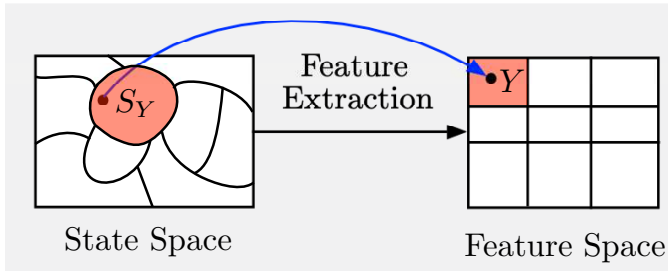

Figure 3.1.7 Feature-based state partitioning using a partition of the space of features. Each set Y of the feature space partition induces a set S_Y of the state space partition that consists of states with "similar" features, i.e., states that map into the same subset of the feature-space partition.

be its optimal cost-to-go when the initial value of the feature vector is v. Then $\hat{J}\big(\phi(x)\big)$ provides an approximation architecture for the original problem, i.e., the architecture that assigns to state x the (exactly) optimal cost-to-go $\hat{J}\big(\phi(x)\big)$ of the feature vector $\phi(x)$ in the aggregate problem. There is considerable freedom on how one formulates and solves aggregate problems. We refer to the DP textbooks [Ber12], [Ber17a], and the RL textbook [Ber19a], Chapter 6, for a detailed treatment; see also the discussion of Section 3.6.

The next example relates to an architecture that is particularly useful when parallel computation is available.

Example 3.1.8 (Feature-Based State Space Partitioning)

A simple method to construct complex and sophisticated approximation architectures, is to partition the state space into several subsets and construct a separate approximation for each subset. For example, by using a separate linear or quadratic polynomial approximation for each subset of the partition, we can construct piecewise linear or piecewise quadratic approximations over the entire state space. Similarly, we may use a separate neural network architecture for each set of the partition. An important issue here is the choice of the method for partitioning the state space. Regular partitions (e.g., grid partitions) may be used, but they often lead to a large number of subsets and very time-consuming computations.

Generally speaking, each subset of the partition should contain "similar" states so that the variation of the optimal cost-to-go over the states of the subset is relatively smooth and can be approximated with smooth functions. An interesting possibility is to use features as the basis for partition. In particular, one may use a more or less regular partition of the space of features, which induces a possibly irregular meaningful partition of the original state space. In this way, each subset of the irregular partition contains states with "similar features;" see Fig. 3.1.7.

As an illustration consider the game of chess. The state here consists of the board position, but the nature of the position evolves through distinct phases: opening, middlegame, and endgame. Each of these phases

may be affected differently by special features of the position. For example there are several different types of endgames (rook endgames, king-and-pawn endgames, minor-piece endgames, etc), which are characterized by identifiable features and call for different playing strategies. It would thus make sense to partition the set of chess positions according to their features, and use a separate strategy on each set of the partition. Indeed this is done to some extent in a number of chess programs.

A potential challenge with partitioned architectures is the discontinuity that can occur at the boundaries between subsets. To address this, a variant known as *soft partitioning*, is sometimes used, whereby the subsets of the partition are allowed to overlap and the discontinuity is smoothed out over their intersection. Once function approximations are computed for each subset, the approximate cost-to-go in the overlapping regions is represented as a smoothly varying linear combination of the corresponding subset approximations.

Partitioning and local approximations can also be used to enhance the quality of approximation in parts of the space where the target function has some special character. The next example addresses one such possibility.

Example 3.1.9 (Local-Global Architectures)

Suppose that the state space is partitioned in subsets $S_1, \ldots, S_M$ and consider approximations of the form

$$\tilde{J}(x,r) = \hat{J}(x,\hat{r}) + \sum_{m=1}^{M} \sum_{k=1}^{K_m} r_m(k)\phi_{k,m}(x), \tag{3.5}$$

where each $\phi_{k,m}(x)$ is a basis function which is local, in the sense that it contributes to the approximation only on the set S_m; that is, it takes the value 0 for $x \notin S_m$. Here $\hat{J}(x,\hat{r})$ is an architecture of the type discussed earlier. The parameter vector r consists of $\hat{r}$ and the coefficients $r_m(k)$ of the basis functions. Thus the portion $\hat{J}(x,\hat{r})$ of the architecture is used to capture "global" aspects of the target function, while each portion

$$\sum_{k=1}^{K_m} r_m(k)\phi_{k,m}(x)$$

is used to capture aspects of the target function that are "local" to the subset S_m.

Note that the training of local-global approximation architectures may potentially be enhanced by using specialized algorithms. The NDP book [BeT96] (Section 3.1.3) discusses computational methods that are tailored to the local-global structure.

Architectures with Automatic Feature Construction

Unfortunately, identifying an adequate set of features can be challenging, so it is important to have methods that construct features automatically, to

supplement any existing ones. Indeed, some architectures are specifically designed to function without pre-existing knowledge of suitable features. The kernel methods of Example 3.1.5 are one such approach. Another very popular possibility is *neural networks*, which we will describe in Section 3.2. Some of these architectures involve training that constructs simultaneously both the feature vectors $\phi(x)$ and the parameter vectors r that weigh them.

Importantly, architectures that generate features automatically can still incorporate additional features based on prior knowledge or problem-specific insights. For example, these architectures can take as input not only the state x, but also supplementary hand-crafted features deemed relevant for the problem. Another possibility is to combine automatically constructed features with other a priori known good features into a (mixed) linear architecture that involves both types of features. The weights of the latter linear architecture may be obtained with a separate second stage training process, following the first stage training process that constructs automatically suitable features using a nonlinear architecture such as a neural network. This approach can enhance the overall effectiveness of the approximation by merging the strengths of automatic feature construction with domain-specific expertise.

3.1.3 Training of Linear and Nonlinear Architectures

In this section, we discuss briefly the training process of choosing the parameter vector r of a parametric architecture $\tilde{J}(x,r)$, focusing primarily on incremental gradient methods. The most common type of training is based on a least squares optimization, also known as *least squares regression*. Here a set of state-cost training pairs

$$(x^s, \beta^s), \qquad s = 1, \ldots, q,$$

called the *training set*, is collected and r is determined by solving the problem

$$\min_r \sum_{s=1}^{q} \big(\tilde{J}(x^s, r) - \beta^s\big)^2. \tag{3.6}$$

Thus r is chosen to minimize the sum of squared errors between the sample costs β^s and the architecture-predicted costs $\tilde{J}(x^s, r)$. Here there is some target cost function J that we aim to approximate with $\tilde{J}(\cdot, r)$, and the sample cost β^s is the value $J(x^s)$ plus perhaps some error or "noise."

The cost function of the training problem (3.6) is generally nonconvex, and can be quite complicated. This may pose challenges, since there may exist multiple local minima. However, for a linear architecture the cost function is convex quadratic, and the training problem admits a closed-form solution. In particular, for the linear architecture

$$\tilde{J}(x,r) = r'\phi(x),$$

the problem becomes

$$\min_r \sum_{s=1}^{q} \big(r'\phi(x^s) - \beta^s\big)^2.$$

By setting the gradient of the quadratic objective to 0, we obtain

$$\sum_{s=1}^{q} \phi(x^s)\big(r'\phi(x^s) - \beta^s\big) = 0,$$

or

$$\sum_{s=1}^{q} \phi(x^s)\phi(x^s)'r = \sum_{s=1}^{q} \phi(x^s)\beta^s.$$

Thus by matrix inversion we obtain the minimizing parameter vector

$$\hat{r} = \left(\sum_{s=1}^{q} \phi(x^s)\phi(x^s)'\right)^{-1} \sum_{s=1}^{q} \phi(x^s)\beta^s. \tag{3.7}$$

If the inverse above does not exist, an additional quadratic in r, called a *regularization* function, is added to the least squares objective to deal with this, and also to help with other issues to be discussed later. A singular value decomposition approach may also be used to deal with the matrix inversion issue; see the NDP book [BeT96], Section 3.2.2.

Thus a linear architecture has the important advantage that the training problem can be solved exactly and conveniently with the formula (3.7) (of course it may be solved by any other algorithm that is suitable for linear least squares problems, including iterative algorithms). By contrast, if we use a nonlinear architecture, such as a neural network, the associated least squares problem is nonquadratic and also nonconvex, so it is hard to solve in principle. Despite this fact, through a combination of sophisticated implementation of special gradient algorithms, called *incremental*, and powerful computational resources, neural network methods have been successful in practice.

Incremental Gradient Methods

We will now discuss briefly special methods for solution of the nonlinear least squares training problem (3.6), assuming a parametric architecture that is differentiable in the parameter vector. This methodology can be properly viewed as a subject in nonlinear programming and iterative algorithms, and as such it can be studied independently of the approximate DP methods of this book. Thus the reader who has already some exposure to the subject may skip to the next section. The author's nonlinear programming textbook [Ber16] and the RL book [Ber19a] provide more detailed presentations.

We view the training problem (3.6) as a special case of the minimization of a sum of component functions

$$f(y) = \sum_{i=1}^{m} f_i(y), \tag{3.8}$$

where each f_i is a differentiable scalar function of the n-dimensional column vector y (this is the parameter vector). Thus we use the more common symbols y and m in place of r and q, respectively, and we replace the squared error terms

$$\big(\tilde{J}(x^s, r) - \beta^s\big)^2$$

in the training problem (3.6) with the generic terms $f_i(y)$.

The (ordinary) gradient method for problem (3.8) generates a sequence $\{y^k\}$ of iterates, starting from some initial guess y^0 for the minimum of the cost function f. It has the form†

$$y^{k+1} = y^k - \gamma^k \nabla f(y^k) = y^k - \gamma^k \sum_{i=1}^{m} \nabla f_i(y^k), \tag{3.9}$$

where γ^k is a positive stepsize parameter. The incremental gradient method is similar to the ordinary gradient method, but uses the gradient of a single component of f at each iteration. It has the general form

$$y^{k+1} = y^k - \gamma^k \nabla f_{i_k}(y^k), \tag{3.10}$$

where i_k is some index from the set $\{1, \ldots, m\}$, chosen by some deterministic or randomized rule. Thus a single component function f_{i_k} is used at iteration k, with great economies in gradient calculation cost over the ordinary gradient method (3.9), particularly when m is large. This is of course a radical simplification, which involves a large approximation error, yet it performs surprisingly well! The idea is to attain faster convergence when far from the solution as we will explain shortly; see the author's books [BeT96], [Ber16], and [Ber19a] for a more detailed discussion.

The method for selecting the index i_k of the component to be iterated on at iteration k is important for the performance of the method. We describe three common rules , the last two of which involve randomization:‡

(1) A *cyclic order*, the simplest rule, whereby the indexes are taken up in the fixed deterministic order $1, \ldots, m$, so that i_k is equal to (k modulo

† We use standard calculus notation for gradients; see, e.g., [Ber16], Appendix A. In particular, $\nabla f(y)$ denotes the n-dimensional column vector whose components are the first partial derivatives $\partial f(y)/\partial y_i$ of f with respect to the components $y_1, \ldots, y_n$ of the column vector y.

‡ With these stepsize rules, the incremental gradient method is often called *stochastic gradient* or *stochastic gradient descent* method.

m) plus 1. A contiguous block of iterations involving the components $f_1, \ldots, f_m$ in this order and exactly once is called a *cycle*.

(2) A *uniform random order*, whereby the index i_k chosen randomly by sampling over all indexes with a uniform distribution, independently of the past history of the algorithm. This rule may perform better than the cyclic rule in some circumstances.

(3) A *cyclic order with random reshuffling*, whereby the indexes are taken up one by one within each cycle, but their order after each cycle is reshuffled randomly (and independently of the past). This rule is used widely in practice, particularly when the number of components m is modest, for reasons to be discussed later.

Note that in the cyclic cases, it is essential to include all components in a cycle; otherwise some components will be sampled more often than others, leading to a bias in the convergence process. Similarly, it is necessary to sample according to the uniform distribution in the random order case.

Focusing for the moment on the cyclic rule (with or without reshuffling), we note that the motivation for the incremental gradient method is faster convergence: we hope that far from the solution, a single cycle of the method will be as effective as several (as many as m) iterations of the ordinary gradient method (think of the case where the components f_i are similar in structure). Near a solution, however, the incremental method may not be as effective.

In particular, we note that there are two complementary performance issues to consider in comparing incremental and nonincremental methods:

(a) *Progress when far from convergence.* Here the incremental method can be much faster. For an extreme case take m large and all components f_i identical to each other. Then an incremental iteration requires m times less computation than a classical gradient iteration, but gives exactly the same result, when the stepsize is scaled to be m times larger. While somewhat extreme, this example reflects the essential mechanism by which incremental methods can be much superior: far from the minimum a single component gradient will point to "more or less" the right direction, at least most of the time; see the following example.

(b) *Progress when close to convergence.* Here the incremental method can be inferior. In particular, the ordinary gradient method (3.9) is convergent with a constant stepsize under reasonable assumptions, see e.g., [Ber16]. However, the incremental method requires a diminishing stepsize, and its ultimate rate of convergence can be much slower.

This behavior is illustrated in the following example, first given in the 1995 first edition of the author's nonlinear programming book [Ber16], and the NDP book [BeT96].

Example 3.1.10

Assume that y is a scalar, and that the problem is

$$\begin{aligned} &\text{minimize} \quad f(y) = \tfrac{1}{2}\sum_{i=1}^{m}(c_i y - b_i)^2 \\ &\text{subject to} \quad y \in \Re, \end{aligned}$$

where c_i and b_i are given scalars with $c_i \neq 0$ for all i. The minimum of each of the components $f_i(y) = \frac{1}{2}(c_i y - b_i)^2$ is

$$y_i^* = \frac{b_i}{c_i},$$

while the minimum of the least squares cost function f is

$$y^* = \frac{\sum_{i=1}^{m} c_i b_i}{\sum_{i=1}^{m} c_i^2}.$$

It can be seen that y^* lies within the range of the component minima

$$R = \left[\min_i y_i^*,\ \max_i y_i^*\right],$$

and that for all y *outside* the range R, the gradient

$$\nabla f_i(y) = c_i(c_i y - b_i)$$

has the same sign as $\nabla f(y)$ (see Fig. 3.1.8). As a result, when outside the region R, the incremental gradient method

$$y^{k+1} = y^k - \gamma^k c_{i_k}(c_{i_k} y^k - b_{i_k})$$

approaches y^* at each step, provided the stepsize γ^k is small enough. In fact it can be verified that it is sufficient that

$$\gamma^k \leq \min_i \frac{1}{c_i^2}.$$

However, for y *inside* the region R, the ith step of a cycle of the incremental gradient method need not make progress. It will approach y^* (for small enough stepsize γ^k) only if the current point y^k does not lie in the interval connecting y_i^* and y^*. This induces an oscillatory behavior within the region R, and as a result, the incremental gradient method will typically not converge to y^* unless $\gamma^k \to 0$. By contrast, the ordinary gradient method, which takes the form

$$y^{k+1} = y^k - \gamma \sum_{i=1}^{m} c_i(c_i y^k - b_i),$$

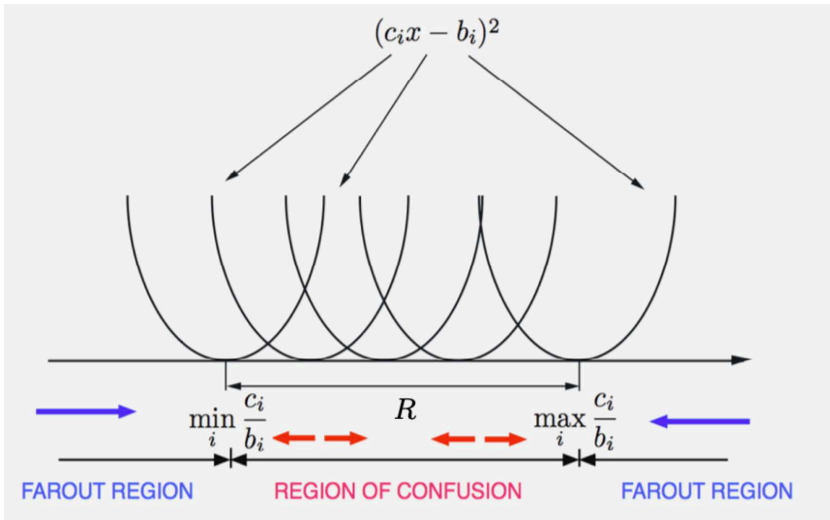

Figure 3.1.8. Illustrating the advantage of incrementalism when far from the optimal solution. The region of component minima

$$R = \left[\min_i y_i^*,\ \max_i y_i^*\right],$$

is labeled as the "region of confusion." It is the region where the method does not have a clear direction towards the optimum. The ith step in an incremental gradient cycle is a gradient step for minimizing $(c_i y - b_i)^2$, so if y lies outside the region of component minima $R = \left[\min_i y_i^*,\ \max_i y_i^*\right]$, (labeled as the "farout region") and the stepsize is small enough, progress towards the solution y^* is made.

can be verified to converge to y^* for any constant stepsize γ with

$$0 < \gamma \leq \frac{1}{\sum_{i=1}^m c_i^2}.$$

However, for y outside the region R, a full iteration of the ordinary gradient method need not make more progress towards the solution than a single step of the incremental gradient method. In other words, with comparably intelligent stepsize choices, *far from the solution (outside R), a single pass through the entire set of cost components by incremental gradient is roughly as effective as m passes by ordinary gradient.*

The preceding example assumes that each component function f_i has a minimum, so that the range of component minima is defined. In cases where the components f_i have no minima, a similar phenomenon may occur, as illustrated by the following example (the idea here is that we may combine several components into a single component that has a minimum).

Example 3.1.11:

Consider the case where f is the sum of increasing and decreasing convex exponentials, i.e.,

$$f_i(y) = a_i e^{b_i y}, \qquad y \in \Re,$$

where a_i and b_i are scalars with $a_i > 0$ and $b_i \neq 0$. Let

$$I^+ = \{i \mid b_i > 0\}, \qquad I^- = \{i \mid b_i < 0\},$$

and assume that I^+ and I^- have roughly equal numbers of components. Let also y^* be the minimum of $\sum_{i=1}^m f_i$.

Consider the incremental gradient method that given the current point, call it y^k, chooses some component f_{i_k} and iterates according to the incremental gradient iteration

$$y^{k+1} = y^k - \alpha^k \nabla f_{i_k}(y^k).$$

Then it can be seen that if $y^k >> y^*$, y^{k+1} will be substantially closer to y^* if $i \in I^+$, and negligibly further away than y^* if $i \in I^-$. The net effect, averaged over many incremental iterations, is that if $y^k >> y^*$, an incremental gradient iteration makes roughly one half the progress of a full gradient iteration, with m times less overhead for calculating gradients. The same is true if $y^k << y^*$. On the other hand as y^k gets closer to y^* the advantage of incrementalism is reduced, similar to the preceding example. In fact in order for the incremental method to converge, a diminishing stepsize is necessary, which will ultimately make the convergence slower than the one of the nonincremental gradient method with a constant stepsize.

The discussion of the preceding examples relies on y being one-dimensional, but in many multidimensional problems the same qualitative behavior can be observed. In particular, a pass through the ith component f_i by the incremental gradient method can make progress towards the solution in the region where the component gradient $\nabla f_{i_k}(y^k)$ makes an angle less than 90 degrees with the cost function gradient $\nabla f(y^k)$. If the components f_i are not "too dissimilar," this is likely to happen in a region of points that are far from the optimal solution set. This behavior has been validated in numerous practical contexts, including the training of neural networks (see the following section), where incremental gradient methods, often referred to as *backpropagation methods*, have been widely employed.

Stepsize Choice

The choice of the stepsize γ^k (often referred to as the *learning rate*) plays an important role in the performance of incremental gradient methods. In practice, it is common to use a constant stepsize for a (possibly prespecified) number of iterations, then decrease the stepsize by a certain factor, and

repeat, up to the point where the stepsize reaches a prespecified floor value. An alternative possibility is to use a diminishing stepsize rule of the form

$$\gamma^k = \min\left\{\gamma,\ \frac{\beta_1}{k+\beta_2}\right\},$$

where γ, β_1, and β_2 are some positive scalars. There are also variants of the method that use a constant stepsize throughout, and can be shown to converge to a stationary point of f under reasonable assumptions. In one type of such method the degree of incrementalism gradually diminishes as the method progresses (see the author's paper [Ber97a]). Another incremental approach with similar aims, is the aggregated gradient method, which is discussed in the author's textbooks [Ber15a], [Ber16], [Ber19a].

Regardless of whether a constant or a diminishing stepsize is ultimately used, the incremental method must use a much larger stepsize than the corresponding nonincremental gradient method (as much as m times larger, so that the size of the incremental gradient step is comparable to the size of the nonincremental gradient step).

One possibility is to use an adaptive stepsize rule, whereby, roughly speaking, the stepsize is reduced (or increased) when the progress of the method indicates that the algorithm is (or is not) oscillating. Formal ways to implement such stepsize rules with sound convergence properties have been proposed by Tseng [Tse98], and Moriyama, Yamashita, and Fukushima [MYF03].

Diagonal Scaling

The difficulty with stepsize selection may also be addressed with *diagonal scaling*, i.e., using a stepsize γ_j^k that is different for each of the components y_j of y. Second derivatives can be very useful for this purpose. A time-honored method in generic nonlinear programming problems of unconstrained minimization of a function f, is to use diagonal scaling with stepsizes

$$\gamma_j^k = \gamma\left(\frac{\partial^2 f(y^k)}{\partial^2 y_j}\right)^{-1}, \qquad j = 1, \ldots, n,$$

where γ is a constant that is nearly equal 1 (the second derivatives may also be approximated by gradient difference approximations). However, in least squares training problems [cf. Eq. (3.6)], this type of scaling is inconvenient because of the additive form of f as a sum of a large number of component functions:

$$f(y) = \sum_{i=1}^{m} f_i(y),$$

cf. Eq. (3.8).

Thus incremental versions of diagonal scaling have been suggested in the neural network literature, and have been incorporated in publicly and commercially available software. Two influential papers are Duchi, Hazan, and Singer [DHS11], and Kingman and Ba [KiB14]. The later paper has introduced the ADAM (Adaptive Moment Estimation) method, which is widely used at present for training neural networks. It adapts the stepsize for each component of the parameter vector r, based on estimates of the first and second moments of the gradient, and incorporates several other ideas that have been proposed in the incremental methods literature over the years.

There are also alternative approaches for scaling the direction of incremental methods to steer it closer to the Newton direction. The RL book [Ber19a] (Section 3.1.3) describes one such method that involves second derivatives, whereby the idea here is to write Newton's method in a format that is well suited to the additive character of the cost function f, and involves low order matrix inversion. One can then implement diagonal scaling by setting to zero the off-diagonal terms of the inverted matrices, so that the algorithm involves no matrix inversion. There is also another related algorithm, which is based on the Gauss-Newton method and the extended Kalman filter; see the author's paper [Ber96], and the books [BeT96] and [Ber16]. Finally, we note an important scaling approach for stochastic parametric optimization problems, which will be discussed in Section 3.5.2.

3.2 NEURAL NETWORKS

There are several different types of neural networks that can be used for a variety of tasks, such as pattern recognition, classification, image and speech recognition, natural language processing, and others. In this section, we focus on our finite horizon DP context, and the role that neural networks can play in approximating the optimal cost-to-go functions J_k^*. As an example within this context, we may first use a neural network to construct an approximation to J_{N-1}^*. Then we may use this approximation to approximate J_{N-2}^*, and continue this process backwards in time, to obtain approximations to all the optimal cost-to-go functions J_k^*, $k = 1, \ldots, N-1$, as we will discuss in more detail in Section 3.3.

Throughout this section, we will focus on the type of neural network, known as a *multilayer perceptron*, which is the one most used at present in the RL applications discussed in this book. Naturally, there are variations that are adapted to the problem at hand. For example AlphaZero uses a specialized neural network that takes advantage of the board-like structure of chess and Go to facilitate and expedite the associated computations.

To describe the use of neural networks in finite horizon DP, let us consider the typical stage k, and for convenience drop the index k; the subsequent discussion applies to each value of k separately. We consider

parametric architectures $\tilde{J}(x, v, r)$ of the form

$$\tilde{J}(x, v, r) = r'\phi(x, v) \tag{3.11}$$

that depend on two parameter vectors v and r. Our objective is to select v and r so that $\tilde{J}(x, v, r)$ approximates some target cost function that can be sampled (possibly with some error). The process is to collect a training set that consists of a large number of state-cost pairs (x^s, β^s), $s = 1, \ldots, q$, and to find a function $\tilde{J}(x, v, r)$ of the form (3.11) that matches the training set in a least squares sense, i.e., (v, r) minimizes

$$\sum_{s=1}^{q} \big(\tilde{J}(x^s, v, r) - \beta^s\big)^2.$$

We postpone for later the question of how the training pairs (x^s, β^s) are generated.† Notice the different roles of the two parameter vectors here: *v parametrizes $\phi(x, v)$*, which in some interpretation may be viewed as a feature vector, and *r is a vector of linear weighting parameters for the components of $\phi(x, v)$*.

Single Layer Perceptron

A neural network architecture provides a parametric class of functions $\tilde{J}(x, v, r)$ of the form (3.11) that can be used in the optimization framework just described. The simplest type of neural network is the *single layer perceptron*; see Fig. 3.2.1. Here the state x is encoded as a vector of numerical values $y(x)$ with components $y_1(x), \ldots, y_n(x)$, which is then transformed linearly as

$$Ay(x) + b,$$

where A is an $m \times n$ matrix and b is a vector in $\Re^m$.‡ This transformation is called the *linear layer* of the neural network. We view the components of A and b as parameters to be determined, and we group them together into the parameter vector $v = (A, b)$.

Each of the m scalar output components of the linear layer,

$$\big(Ay(x) + b\big)_\ell, \qquad \ell = 1, \ldots, m,$$

† The least squares training problem used here is based on *nonlinear regression*. This is a classical method for approximating the expected value of a function with a parametric architecture, and involves a least squares fit of the architecture to simulation-generated samples of the expected value. We refer to machine learning and statistics textbooks for more discussion.

‡ The method of encoding x into the numerical vector $y(x)$ is generally problem-dependent, but it can be critical for the success of the training process. We should note also that some of the components of $y(x)$ could be known interesting features of x that can be designed based on problem-specific knowledge.

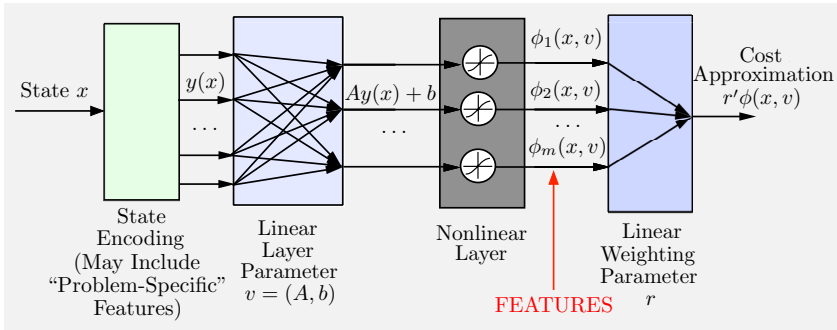

Figure 3.2.1 Schematic illustration of a single layer perceptron, a neural network consisting of a linear layer and a nonlinear layer. It provides a way to compute features of the state, which can be used for cost function approximation. The state x is encoded as a vector of numerical values $y(x)$, which is then transformed linearly as $Ay(x) + b$ in the linear layer. The m scalar output components of the linear layer, become the inputs to nonlinear one-dimensional functions $\sigma : \Re \mapsto \Re$, thus producing the m scalars

$$\phi_\ell(x, v) = \sigma\big((Ay(x) + b)_\ell\big),$$

which can be viewed as features that are in turn linearly weighted with parameters r_ℓ.

becomes the input to a nonlinear differentiable and monotonically increasing function σ that maps scalars to scalars. A simple and popular possibility is the *rectified linear unit* (ReLU for short), which is simply the function $\max\{0, \xi\}$, approximated by a differentiable function σ by some form of smoothing operation; for example $\sigma(\xi) = \ln(1 + e^\xi)$, which is illustrated in Fig. 3.2.2. Other functions, used since the early days of neural networks, have the property

$$-\infty < \lim_{\xi \to -\infty} \sigma(\xi) < \lim_{\xi \to \infty} \sigma(\xi) < \infty;$$

see Fig. 3.2.3. Such functions are called *sigmoids*, and some common choices are the *hyperbolic tangent* function

$$\sigma(\xi) = \tanh(\xi) = \frac{e^\xi - e^{-\xi}}{e^\xi + e^{-\xi}},$$

and the *logistic* function

$$\sigma(\xi) = \frac{1}{1 + e^{-\xi}}.$$

In what follows, we will ignore the character of the function σ (except for differentiability), and simply refer to it as a "nonlinear unit" and to the corresponding layer as a "nonlinear layer."

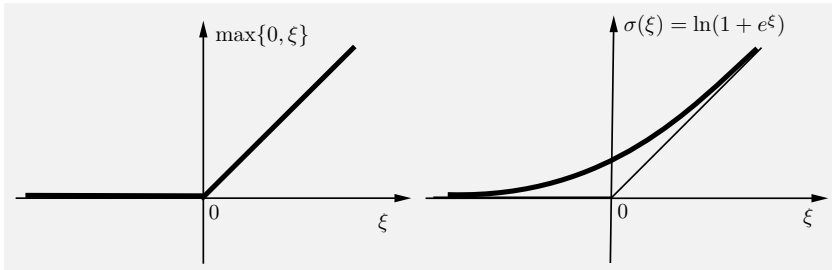

Figure 3.2.2 The rectified linear unit $\sigma(\xi) = \ln(1 + e^{\xi})$. It is the function $\max\{0, \xi\}$ with its corner "smoothed out." Its derivative is $\sigma'(\xi) = e^{\xi}/(1 + e^{\xi})$, and approaches 0 and 1 as $\xi \to -\infty$ and $\xi \to \infty$, respectively.

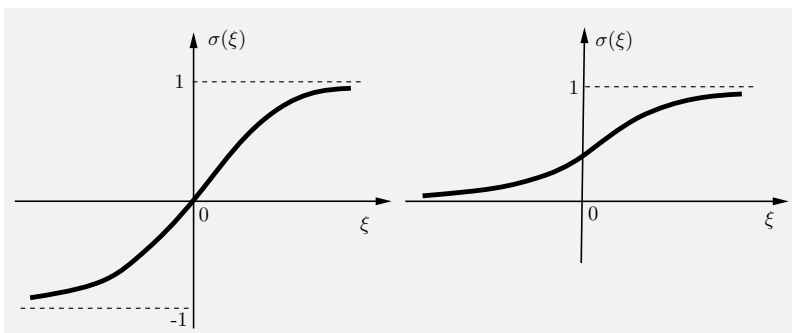

Figure 3.2.3 Some examples of sigmoid functions. The hyperbolic tangent function is on the left, while the logistic function is on the right.

At the outputs of the nonlinear units, we obtain the scalars

$$\phi_\ell(x, v) = \sigma\big((Ay(x) + b)_\ell\big), \qquad \ell = 1, \ldots, m.$$

One possible interpretation is to view $\phi_\ell(x, v)$ as features of x, which are linearly combined using weights r_ℓ, $\ell = 1, \ldots, m$, to produce the final output

$$\tilde{J}(x, v, r) = \sum_{\ell=1}^{m} r_\ell \phi_\ell(x, v) = \sum_{\ell=1}^{m} r_\ell \sigma\big((Ay(x) + b)_\ell\big).$$

Note that each value $\phi_\ell(x, v)$ depends on just the ℓth row of A and the ℓth component of b, not on the entire vector v. In some cases this motivates placing some constraints on individual components of A and b to achieve special problem-dependent "handcrafted" effects.

State Encoding and Direct Feature Extraction

The state encoding operation that transforms x into the neural network input $y(x)$ can be instrumental in the success of the approximation scheme.

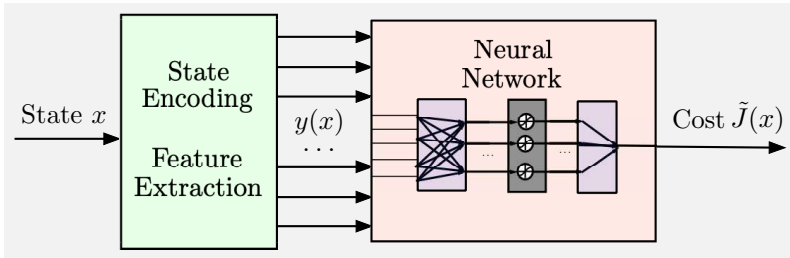

Figure 3.2.4 A nonlinear architecture with a view of the state encoding process as a feature extraction mapping preceding the neural network. The state encoder may also contain tunable parameters.

Examples of state encodings are components of the state x, numerical representations of qualitative characteristics of x, and more generally features of x, i.e., functions of x that aim to capture "important nonlinearities" of the optimal cost-to-go function. With the latter view of state encoding, we may consider the approximation process as consisting of a feature extraction mapping, followed by a neural network with input the extracted features of x, and output the cost-to-go approximation; see Fig. 3.2.4.

The idea here is that with a well-designed feature extraction mapping, the neural network need not be very complicated and may be trained more easily. This intuition is borne out by simple examples and practical experience. However, as is often the case with neural networks, it is hard to support it with a quantitative analysis.

Universal Approximation Property of Neural Networks

An important question is how well we can approximate the target function J_k^* with a neural network architecture, assuming we can choose the number of the nonlinear units m to be as large as we want. The answer to this question is quite favorable and is provided by the so-called *universal approximation theorem*.

Roughly, the theorem says that assuming that x is an element of a Euclidean space X and $y(x) \equiv x$, a neural network of the form described can approximate arbitrarily closely (in an appropriate mathematical sense), over a compact subset $S \subset X$, any piecewise continuous function $J : S \mapsto \Re$, provided the number m of nonlinear units is sufficiently large. For proofs of the theorem, we refer to Cybenko [Cyb89], Funahashi [Fun89], Hornik, Stinchcombe, and White [HSW89], and Leshno et al. [LLP93]. For additional sources and intuitive explanations we refer to Bishop ([Bis95], pp. 129-130), Jones [Jon90], and the RL textbook [Ber19a], Section 3.2.1.

While the universal approximation theorem provides some reassurance about the adequacy of the neural network structure, it does not predict how many nonlinear units we may need for "good" performance in a given problem. Unfortunately, this is a difficult question to even pose